

Základní řídicí struktury

Příprava k maturitě - SPŠT IT

Obsah

1 Pojem algoritmus	2
2 Vlastnosti algoritmu	2
3 Způsoby zápisu algoritmu	2
4 Strukturované programování	2
5 Sekvence	2
6 Selekce (větvení)	3
6.1 Příkaz if	3
6.2 Příkaz switch	4
7 Iterace (cyklus)	4
7.1 Cyklus while	4
7.2 Cyklus do-while	5
7.3 Cyklus for	5
8 Generování náhodných čísel	5
8.1 C++	5
8.2 C#	6
9 Kolekce Controls v C#	6
9.1 Základní použití	6
9.2 Přístup k prvkům podle typu	6
9.3 Přístup podle jména	7
9.4 Rekurzivní procházení	7
Odpovědi na zadané podotázky	7
Další materiály a zdroje	8

1 Pojem algoritmus

Algoritmus je přesný návod (postup) jak řešit určitý problém. Jde o konečnou posloupnost jasně definovaných kroků, které vedou k vyřešení úlohy.

2 Vlastnosti algoritmu

1. **Elementárnost** - algoritmus se skládá z konečného počtu jednoduchých (elementárních) kroků
2. **Konečnost** (finitnost) - algoritmus musí skončit v konečném počtu kroků
3. **Obecnost** - algoritmus řeší celou třídu úloh, ne pouze jeden konkrétní případ
4. **Determinovanost** - každý krok je jednoznačně definován, při opakovaném provedení se stejnými vstupními daty dostaneme stejné výsledky
5. **Rezultativnost** - algoritmus vždy dospěje k výsledku

3 Způsoby zápisu algoritmu

1. **Slovní popis** - popis v přirozeném jazyce (čeština, angličtina)
2. **Vývojový diagram** - grafické znázornění pomocí symbolů a šipek
3. **Strukturogram** (Nassi-Shneidermanův diagram) - blokové schéma bez použití šipek
4. **Pseudokód** - zápis podobný programovacímu jazyku, ale zjednodušený a srozumitelný
5. **Programovací jazyk** - zápis v konkrétním programovacím jazyce (C++, C#, Python...)

4 Strukturované programování

Programovací paradigma, které klade důraz na:

- Přehlednost a čitelnost kódu
- Používání pouze tří základních řídicích struktur: **sekvence**, **selekce**, **iterace**
- Vyhýbání se příkazu GOTO
- Dekompozici problému na menší části (procedury, funkce)
- Jednodušší ladění a údržbu kódu

5 Sekvence

Sekvence je základní řídicí struktura, kde se příkazy vykonávají postupně jeden za druhým v pořadí, jak jsou napsány.

cpp

```
1 // C++
2 int a = 5;
```

```
3 int b = 10;
4 int soucet = a + b;
5 cout << soucet << endl;
```

cs

```
1 // C#
2 int a = 5;
3 int b = 10;
4 int soucet = a + b;
5 Console.WriteLine(soucet);
```

6 Selekce (větvení)

Umožňuje podmíněné vykonání příkazů na základě splnění podmínky.

6.1 Příkaz if

C++:

cpp

```
1 if (podminka) {
2     // kód, který se vykoná, když je podmínka splněna
3 }
4
5 if (podminka) {
6     // kód pro splněnou podmínku
7 } else {
8     // kód pro nesplněnou podmínku
9 }
10
11 if (podminka1) {
12     // kód 1
13 } else if (podminka2) {
14     // kód 2
15 } else {
16     // kód 3
17 }
```

C#:

cs

```
1 if (podminka) {
2     // kód, který se vykoná, když je podmínka splněna
3 }
4
5 if (podminka) {
6     // kód pro splněnou podmínku
7 } else {
8     // kód pro nesplněnou podmínku
9 }
10
11 if (podminka1) {
12     // kód 1
13 } else if (podminka2) {
14     // kód 2
15 } else {
```

```
16 // kód 3
17 }
```

6.2 Příkaz switch

Používá se pro vícenásobné větvení na základě hodnoty výrazu.

C++:

cpp

```
1 switch (promenna) {
2     case hodnota1:
3         // kód
4         break;
5     case hodnota2:
6         // kód
7         break;
8     default:
9         // kód pro ostatní případy
10        break;
11 }
```

C#:

cs

```
1 switch (promenna) {
2     case hodnota1:
3         // kód
4         break;
5     case hodnota2:
6         // kód
7         break;
8     default:
9         // kód pro ostatní případy
10        break;
11 }
```

7 Iterace (cyklus)

Umožňuje opakované vykonávání bloku příkazů.

7.1 Cyklus while

Cyklus s podmínkou na začátku – testuje podmínku před každou iterací.

C++:

cpp

```
1 while (podminka) {
2     // kód, který se opakuje, dokud je podmínka splněna
3 }
```

C#:

cs

```
1 while (podminka) {
2     // kód, který se opakuje, dokud je podmínka splněna
```

```
3 }
```

7.2 Cyklus do-while

Cyklus s podmínkou na konci – tělo cyklu se provede vždy alespoň jednou.

C++:

cpp

```
1 do {
2     // kód, který se vykoná alespoň jednou
3 } while (podminka);
```

C#:

cs

```
1 do {
2     // kód, který se vykoná alespoň jednou
3 } while (podminka);
```

7.3 Cyklus for

Cyklus s řídicí proměnnou – používá se, když známe počet opakování.

C++:

cpp

```
1 for (int i = 0; i < 10; i++) {
2     // kód, který se opakuje 10x
3 }
```

C#:

cs

```
1 for (int i = 0; i < 10; i++) {
2     // kód, který se opakuje 10x
3 }
```

8 Generování náhodných čísel

8.1 C++

cpp

```
1 #include <cstdlib> // pro rand() a srand()
2 #include <ctime>   // pro time()
3
4 // Inicializace generátoru (provést jednou na začátku programu)
5 srand(time(0));
6
7 // Generování náhodného čísla
8 int nahodneCislo = rand(); // 0 až RAND_MAX
9
10 // Náhodné číslo v rozsahu 0-99
11 int cislo = rand() % 100;
12
13 // Náhodné číslo v rozsahu 1-100
14 int cislo = rand() % 100 + 1;
```

```

15
16 // Náhodné číslo v rozsahu min-max
17 int cislo = rand() % (max - min + 1) + min;

```

8.2 C#

CS

```

1 // Vytvoření instance Random (provést jednou)
2 Random rnd = new Random();
3
4 // Generování náhodného čísla
5 int nahodneCislo = rnd.Next(); // nezáporné celé číslo
6
7 // Náhodné číslo 0-99
8 int cislo = rnd.Next(100);
9
10 // Náhodné číslo 1-100
11 int cislo = rnd.Next(1, 101); // horní mez je exkluzivní
12
13 // Náhodné desetinné číslo 0.0-1.0
14 double des = rnd.NextDouble();

```

9 Kolekce Controls v C#

Kolekce `Controls` je vlastnost kontejnerových prvků (Form, Panel, GroupBox...) ve Windows Forms, která obsahuje všechny ovládací prvky (controls) umístěné v daném kontejneru.

9.1 Základní použití

CS

```

1 // Přístup ke všem ovládacím prvkům formuláře
2 foreach (Control ctrl in this.Controls) {
3     // Práce s jednotlivými prvky
4 }
5
6 // Přidání ovládacího prvku
7 Button btn = new Button();
8 btn.Text = "Nové tlačítko";
9 this.Controls.Add(btn);
10
11 // Odebrání ovládacího prvku
12 this.Controls.Remove(btn);
13
14 // Počet prvků v kolekci
15 int pocet = this.Controls.Count;

```

9.2 Přístup k prvkům podle typu

CS

```

1 // Najít všechny TextBoxy
2 foreach (Control ctrl in this.Controls) {
3     if (ctrl is TextBox) {
4         TextBox tb = (TextBox)ctrl;

```

```

5         tb.Clear(); // Vymazání obsahu
6     }
7 }
8
9 // Pomocí LINQ
10 var textBoxy = this.Controls.OfType<TextBox>();
11 foreach (TextBox tb in textBoxy) {
12     tb.Clear();
13 }

```

9.3 Přístup podle jména

CS

```

1 // Najít prvek podle jména
2 Control ctrl = this.Controls["textBox1"];
3 if (ctrl != null && ctrl is TextBox) {
4     TextBox tb = (TextBox)ctrl;
5     tb.Text = "Nový text";
6 }
7
8 // Nebo pomocí Find
9 Control[] controls = this.Controls.Find("textBox1", true);
10 if (controls.Length > 0) {
11     TextBox tb = (TextBox)controls[0];
12 }

```

9.4 Rekurzivní procházení

Pro práci s vnořenými kontejnery (Panel v Panelu atd.):

CS

```

1 private void ProcházejVšechnyPrvky(Control kontejner) {
2     foreach (Control ctrl in kontejner.Controls) {
3         // Zpracování prvku
4         Console.WriteLine(ctrl.Name);
5
6         // Rekurzivní volání pro vnořené kontejnery
7         if (ctrl.HasChildren) {
8             ProcházejVšechnyPrvky(ctrl);
9         }
10    }
11 }
12
13 // Použití
14 ProcházejVšechnyPrvky(this);

```

Odpovědi na zadané podotázky

Každá otázka na začátku odpovědi obsahuje odkaz na konkrétní kapitolu v zápise s proklikem

Další materiály a zdroje

Žádné další zdroje
